1. Mathematical & Acoustic Physics Foundations

Synthesizing unvoiced aspirated stops (/p, t, k/) and nasal consonants (/m, n/) requires expanding the source-filter framework to accommodate **asymmetric aerodynamic voicing onset (VOT)**, **glottal turbulent aspiration noise**, and **branched acoustic stub antiresonances (zeroes)**.

```
                    UNVOICED ASPIRATED STOP (/p, t, k/)
Time:     t = 0..50 ms         t = 50..60 ms        t = 60..110 ms (VOT)     t > 110 ms
Phase:    [ Silent Closure ] ──► [ Release Burst ] ──► [ Aspiration Noise ] ──► [ Voiced Vowel ]
Acoustic:  Total Silence         Transient Shock      Glottal Turbulence      Periodic Glottal
           (No Voice Bar)        Shaped Resonator     Open Tract Filter       Formant Resonators

                          NASAL MURMUR & TRANSITION (/m, n/)
Acoustic  [ Closed Oral Cavity: Stub Length ℓ_stub ] ──► Antiformant Zero Z₁ = c / (4 ℓ_stub)
Coupling: [ Open Velopharyngeal Port (Nasal Cavity) ] ──► Low Nasal Formant F_N1 ≈ 250–300 Hz
```

1.1 Unvoiced Aspirated Stops (/p, t, k/): Burst & Aspiration Mechanics

Unlike voiced stops (/b, d, g/) which exhibit a low-frequency pre-voicing lead (voice bar), unvoiced stops feature:

1. **Silent Occlusion:** Complete oral and velopharyngeal closure with no transglottal airflow ($U_g = 0$).
2. **Release Burst:** A sudden release of built-up intraoral pressure ($P_0 \approx 600\text{--}1000\ \text{Pa}$) producing a short shock impulse ($5\text{--}15\ \text{ms}$) filtered by the cavity forward of the constriction.
3. **Aspiration Interval (Long Voice Onset Time - VOT):** The vocal folds remain abducted (open) post-release for $40\text{--}80\ \text{ms}$. High-velocity turbulent air passing through the open glottis generates wideband aspiration noise $w_{\text{asp}}(t)$, which excites the time-varying vocal tract resonators before periodic voicing begins:

$$e_{\text{asp}}(t) = A_{\text{asp}}(t) \cdot [w(t) * h_{\text{tilt}}(t)]$$

where $h_{\text{tilt}}(t)$ applies a $-6\ \text{dB/octave}$ spectral tilt modeling glottal turbulence damping.

1.2 Nasal Murmurs (/m, n/): Acoustic Stub Theory & Antizeroes

During a nasal murmur, the velopharyngeal port is open, coupling the pharynx to the nasal cavity ($L_{\text{nasal}} \approx 12\text{--}14\ \text{cm}$), while the oral cavity is occluded at the lips (/m/) or the alveolar ridge (/n/).

The closed oral cavity acts as a **side-branch acoustic stub filter** terminating in infinite impedance. At quarter-wavelength standing wave frequencies of the oral stub length ℓ_{stub}, acoustic volume velocity is trapped within the mouth, creating **transmission zeroes (antiformants)** in the radiated nasal spectrum:

$$Z_k = \frac{(2k-1)c}{4\ell_{\text{stub}}}, \quad k = 1, 2, \dots$$

- **/m/ (Bilabial Closure):** Long oral cavity stub ($\ell_{\text{stub}} \approx 7.0\text{--}8.5\ \text{cm}$) $\implies$ First Antizero $Z_1 \approx 800\text{--}1100\ \text{Hz}$.
- **/n/ (Alveolar Closure):** Shorter oral cavity stub ($\ell_{\text{stub}} \approx 4.0\text{--}5.5\ \text{cm}$) $\implies$ First Antizero $Z_1 \approx 1500\text{--}1900\ \text{Hz}$.

The nasalized transfer function $H_{\text{nasal}}(s)$ combines the primary nasal murmur pole ($F_{N1} \approx 250\text{--}300\ \text{Hz}$), oral formants F_k, and the oral antizero Z_1:

$$H_{\text{nasal}}(s) = \frac{s^2 + 2\pi B_{z1} s + (2\pi Z_1)^2}{s^2 + 2\pi B_{n1} s + (2\pi F_{N1})^2} \prod_{k=1}^{M} \frac{(2\pi F_k)^2}{s^2 + 2\pi B_k s + (2\pi F_k)^2}$$

---

2. Acoustic Parameter Matrix (Stops & Nasals)

Phoneme	Class	Closure Duration	Burst Center (F_b)	VOT / Aspiration	F_1 Locus	F_2 Locus	F_3 Locus	Antizero (Z_1)	Nasal Pole (
/p/	Unvoiced Bilabial Stop	$50\ \text{ms}$ (Silent)	$800\ \text{Hz}$ (Flat)	$50\ \text{ms}$ (High)	$180\ \text{Hz}$	$750\ \text{Hz}$	$2100\ \text{Hz}$	None	None
/t/	Unvoiced Alveolar Stop	$45\ \text{ms}$ (Silent)	$4200\ \text{Hz}$ (Sharp)	$60\ \text{ms}$ (High)	$200\ \text{Hz}$	$1800\ \text{Hz}$	$2700\ \text{Hz}$	None	None
/k/	Unvoiced Velar Stop	$55\ \text{ms}$ (Silent)	$1800\ /\ 2600\ \text{Hz}$	$70\ \text{ms}$ (High)	$220\ \text{Hz}$	$2400\ /\ 1400\ \text{Hz}$	$2300\ \text{Hz}$	None	None

Phoneme	Class	Closure Duration	Burst Center (F_b)	VOT / Aspiration	F_1 Locus	F_2 Locus	F_3 Locus	Antizero (Z_1)	Nasal Pole
/m/	Bilabial Nasal	80 ms (Voiced)	None	0 ms	250 Hz	1000 Hz	2200 Hz	950 Hz	280 Hz (B = 1
/n/	Alveolar Nasal	75 ms (Voiced)	None	0 ms	280 Hz	1700 Hz	2600 Hz	1750 Hz	290 Hz (B = 1

3. Complete Python Implementation: Stops, Nasals & Antizeroes

```python
import numpy as np
import scipy.signal as signal
import wave

class AdvancedSpeechSynthesizer:
    """
    Parametric articulatory speech synthesizer supporting:
      - Voiced & unvoiced aspirated stops (/p, t, k, b, d, g/) with release bursts & VOT aspiration
      - Nasal murmurs (/m, n/) with acoustic stub antiformants (zeroes) & velopharyngeal coupling
    """
    def __init__(self, sample_rate: int = 44100):
        self.sr = sample_rate

    def _rosenberg_pulse_deriv(self, f0: float, duration: float) -> np.ndarray:
        """Generates differentiated Rosenberg glottal pulse train for lip radiation."""
        N = int(self.sr * duration)
        pulse = np.zeros(N)
        period = int(self.sr / f0)
        t_open = int(0.40 * period)
        t_close = int(0.16 * period)

        for p in range(0, N, period):
            for i in range(period):
                idx = p + i
                if idx >= N:
                    break
                if i < t_open:
                    pulse[idx] = 0.5 * (1.0 - np.cos(np.pi * i / t_open))
                elif i < t_open + t_close:
                    pulse[idx] = np.cos(np.pi * (i - t_open) / (2.0 * t_close))
                else:
                    pulse[idx] = 0.0

        d_pulse = np.diff(pulse, prepend=0)
        return d_pulse / (np.max(np.abs(d_pulse)) + 1e-9)

    def _biquad_pole(self, x: np.ndarray, f_traj: np.ndarray, bw_traj: np.ndarray) -> np.ndarray:
        """Applies time-varying 2nd-order resonant pole (Formant Resonator)."""
        N = len(x)
        y = np.zeros(N)
        w1 = 0.0
        w2 = 0.0

        for n in range(N):
            f = np.clip(f_traj[n], 20.0, self.sr * 0.49)
            bw = np.clip(bw_traj[n], 10.0, self.sr * 0.25)
            r = np.exp(-np.pi * bw / self.sr)
            theta = 2.0 * np.pi * f / self.sr

            a1 = -2.0 * r * np.cos(theta)
            a2 = r * r
            b0 = 1.0 - r

            x_n = x[n]
            y_n = b0 * x_n + w1
            w1 = -a1 * y_n + w2
            w2 = -a2 * y_n
            y[n] = y_n
```

```
        return y

    def _biquad_zero_notch(self, x: np.ndarray, f_zero_traj: np.ndarray,
    bw_zero: float = 120.0, depth_traj: np.ndarray = None) -> np.ndarray:
        """
        Applies time-varying 2nd-order antiresonant zero (Acoustic Stub Notch
        Filter).
        depth_traj: 1.0 during nasal murmur (full notch), fading to 0.0 (no
        zero).
        """
        N = len(x)
        y = np.zeros(N)
        if depth_traj is None:
            depth_traj = np.ones(N)

        for n in range(N):
            depth = depth_traj[n]
            if depth < 1e-3:
                y[n] = x[n]
                continue

            fz = np.clip(f_zero_traj[n], 50.0, self.sr * 0.48)
            # Notch filter design
            r_z = 0.985  # Zero radius close to unit circle
            r_p = r_z - (bw_zero / self.sr) * np.pi * depth
            theta = 2.0 * np.pi * fz / self.sr

            # Numerator (Zero) and Denominator (Pole for selective notch
            bandwidth)
            b0 = 1.0
            b1 = -2.0 * r_z * np.cos(theta)
            b2 = r_z * r_z

            a1 = -2.0 * r_p * np.cos(theta)
            a2 = r_p * r_p

            # Direct calculation with depth weighting
            x_n = x[n]
            y_notch = (b0 * x_n + b1 * (x[n-1] if n > 0 else 0) + b2 * (x[n-2]
            if n > 1 else 0)
                       - a1 * (y[n-1] if n > 0 else 0) - a2 * (y[n-2] if n > 1
                       else 0))

            y[n] = (1.0 - depth) * x_n + depth * y_notch
        return y

    def synthesize(self, phoneme: str = 'p', vowel: str = 'a', f0: float = 120.
    0, duration: float = 0.40) -> np.ndarray:
        N = int(self.sr * duration)
        t = np.linspace(0, duration, N, endpoint=False)

        # Vowel Formants (F1, F2, F3, F4) & Bandwidths (BW1..4)
        vowel_db = {
            'a': {'F': [730, 1090, 2440, 3400], 'BW': [80, 90, 130, 180]},
            'i': {'F': [270, 2290, 3010, 3500], 'BW': [50, 100, 150, 200]},
            'u': {'F': [300, 870, 2240, 3400],  'BW': [60, 80, 110, 160]}
        }
        v_target = vowel_db.get(vowel, vowel_db['a'])

        # Phoneme Articulatory Specifications
        consonant_specs = {
            'p': {'type': 'unvoiced_stop', 'locus': [180, 750, 2100, 3400],
            'burst_f': 800,  'burst_bw': 500, 'burst_a': 0.35, 'vot': 0.055,
            'closure': 0.045},
            't': {'type': 'unvoiced_stop', 'locus': [200, 1800, 2700, 3400],
            'burst_f': 4200, 'burst_bw': 800, 'burst_a': 0.55, 'vot': 0.065,
            'closure': 0.040},
            'k': {'type': 'unvoiced_stop', 'locus': [220, 2300 if vowel == 'i'
            else 1400, 2300, 3400],
                  'burst_f': 2400 if vowel == 'i' else 1700, 'burst_bw': 400,
                  'burst_a': 0.60, 'vot': 0.075, 'closure': 0.050},
            'm': {'type': 'nasal', 'locus': [250, 1000, 2200, 3400], 'z1':
```

```
            950, 'fn1': 280, 'closure': 0.080, 'trans': 0.035},
        'n': {'type': 'nasal', 'locus': [280, 1700, 2600, 3400], 'z1':
        1750, 'fn1': 290, 'closure': 0.075, 'trans': 0.035},
    }

    c_spec = consonant_specs.get(phoneme.lower(), consonant_specs['p'])
    t_closure = c_spec['closure']
    t_rel = t_closure

    # 1. Source Excitation Generation
    glottal_raw = self._rosenberg_pulse_deriv(f0, duration)
    excitation = np.zeros(N)

    if c_spec['type'] == 'unvoiced_stop':
        vot = c_spec['vot']
        t_vot_end = t_rel + vot

        # Aspiration Noise (Turbulence through open glottis)
        noise = np.random.normal(0, 1, N)
        sos_tilt = signal.butter(1, 1500, 'lowpass', fs=self.sr,
        output='sos')
        asp_noise = signal.sosfilt(sos_tilt, noise) * 0.25

        # Transient Release Burst
        sos_burst = signal.butter(2, [max(100, c_spec['burst_f'] - c_spec
        ['burst_bw']/2),
                                      min(self.sr*0.48, c_spec['burst_f'] +
                                      c_spec['burst_bw']/2)],
                                  'bandpass', fs=self.sr, output='sos')
        burst_raw = signal.sosfilt(sos_burst, np.random.normal(0, 1, N))

        for n in range(N):
            tn = t[n]
            if tn < t_rel:
                excitation[n] = 0.0  # Silent occlusion
            elif t_rel <= tn < t_rel + 0.012:
                # Burst spike
                dt = tn - t_rel
                env = (dt / 0.002) * np.exp(1.0 - dt / 0.002)
                excitation[n] = burst_raw[n] * env * c_spec['burst_a']
            elif t_rel + 0.012 <= tn < t_vot_end:
                # Aspiration window
                asp_env = np.sin(np.pi * (tn - (t_rel + 0.012)) / (vot - 0.
                012))
                excitation[n] = asp_noise[n] * asp_env
            else:
                # Voicing onset ramp
                v_ramp = np.clip((tn - t_vot_end) / 0.02, 0.0, 1.0)
                excitation[n] = glottal_raw[n] * v_ramp

    elif c_spec['type'] == 'nasal':
        # Continuous voicing: heavily damped during nasal murmur, opening
        post-release
        for n in range(N):
            tn = t[n]
            if tn < t_rel:
                excitation[n] = glottal_raw[n] * 0.65  # Murmur volume
                velocity
            else:
                excitation[n] = glottal_raw[n]

    # 2. Formant Trajectories & Filter Application
    f_trajs = np.zeros((4, N))
    bw_trajs = np.zeros((4, N))
    tau_trans = 0.018

    for k in range(4):
        f_locus = c_spec['locus'][k]
        f_vowel = v_target['F'][k]
        bw_v = v_target['BW'][k]

        for n in range(N):
            tn = t[n]
```

```
                    if tn < t_rel:
                        f_trajs[k, n] = f_locus
                        bw_trajs[k, n] = bw_v * (1.8 if c_spec['type'] == 'nasal'
                        else 1.0)
                    else:
                        dt = tn - t_rel
                        f_trajs[k, n] = f_vowel + (f_locus - f_vowel) * np.exp
                        (-dt / tau_trans)
                        bw_trajs[k, n] = bw_v

            # 3. Resonant Formant Filtering (F1..F4)
            audio = total_signal = excitation
            for k in range(4):
                audio = self._biquad_pole(audio, f_trajs[k], bw_trajs[k])

            # 4. Nasal Antizero & Nasal Pole (F_N1) Filtering for Nasals
            if c_spec['type'] == 'nasal':
                # Low nasal murmur pole F_N1
                fn1_traj = np.full(N, c_spec['fn1'])
                bw_n1 = np.full(N, 100.0)
                nasal_pole_audio = self._biquad_pole(excitation, fn1_traj, bw_n1) *
                0.85

                # Antizero notch depth envelope (1.0 during closure, fading during
                transition)
                depth_traj = np.zeros(N)
                z1_traj = np.full(N, c_spec['z1'])

                for n in range(N):
                    if t[n] < t_rel:
                        depth_traj[n] = 1.0
                    elif t[n] < t_rel + c_spec['trans']:
                        depth_traj[n] = 1.0 - ((t[n] - t_rel) / c_spec['trans'])
                    else:
                        depth_traj[n] = 0.0

                # Apply oral stub notch filter
                audio = self._biquad_zero_notch(audio, z1_traj, bw_zero=110.0,
                depth_traj=depth_traj)
                audio = audio + (nasal_pole_audio * depth_traj)

            # 5. Normalization
            audio = audio - np.mean(audio)
            audio = audio / (np.max(np.abs(audio)) + 1e-9)
            return audio


    # Verification Script
    if __name__ == "__main__":
        synth = AdvancedSpeechSynthesizer(sample_rate=44100)
        syllables = [('p', 'a'), ('t', 'a'), ('k', 'a'), ('m', 'a'), ('n', 'a')]

        for c, v in syllables:
            pcm = synth.synthesize(c, v, f0=125.0, duration=0.35)
            out_name = f"synth_{c}{v}.wav"

            with wave.open(out_name, 'wb') as wf:
                wf.setnchannels(1)
                wf.setsampwidth(2)
                wf.setframerate(44100)
                wf.writeframes(np.int16(pcm * 32767 * 0.9).tobytes())

            print(f"Generated syllable /{c}{v}/ -> Output written to '{out_name}'")
```

```
Generated syllable /pa/ -> Output written to 'synth_pa.wav'
Generated syllable /ta/ -> Output written to 'synth_ta.wav'
Generated syllable /ka/ -> Output written to 'synth_ka.wav'
Generated syllable /ma/ -> Output written to 'synth_ma.wav'
Generated syllable /na/ -> Output written to 'synth_na.wav'
```

4. Interactive Web Audio Visualizer & Syllable Synthesizer

The interactive workspace below couples the release burst generator, turbulent aspiration noise, and nasal antizero notch filter with real-time Web Audio API playback and live spectral locus tracing.
